freshtime: two time tools, one contract

How freshtime-core, freshtime-intf, freshtime-agnt and pi-time fit together on branch `feat/pi-time-poc`, what the six test layers show, and what to build next.

PROJECT
`freshtime`

DATE
2026-10-22

STATUS
Draft

BRANCH
`feat/pi-time-poc`

COMMIT
`e9ef5f7`

FOR
Jeremy Roethel, project owner.

STATE
branch `feat/pi-time-poc` at commit `e9ef5f7`, 2026-10-22.

NUMBERS
Re-run for this page on 2026-10-22, except the replay and end-to-end suites, which are read from their latest results (Section 06).

BACKGROUND
The "Two tools, one contract" and "Gaps" sections of the October 2026 memory-systems research page on time coverage.

CONTENT MASTER
`docs/content/freshtime-design/content.md`.

A

THE ANSWER

freshtime is two small time tools that share one contract, and on the synthetic test the pair shows every fact with the right status and nothing false as true, which neither tool does alone.

freshtime-intf reads which time a question is about: now, a past date, a span, the future, or a past date plus what changed since. freshtime-agnt keeps dated facts from project docs, knows when each one goes stale, and re-reads the source before it shows a stale fact as current. pi-time is pi launched with both tools in its own agent directory, over a ledger built from project docs.

The 1+1=3 test runs seven questions under four setups: no tools, intent only, ledger only, and both. Status accuracy adds up: from a 36% baseline, intent alone gains 43 points, the ledger alone 21, and both together 64, so the interaction term is 0.00. Harm falls by more than the sum: intent alone removes 2 of the 4 false-as-true facts, the ledger alone removes none, and both together remove all 4, an interaction of -2. With a fresh ledger per question the pair spends 9 source re-reads where the ledger alone spends 24. The fixtures are synthetic, so this shows the mechanism and not a rate on real data. On 21 contrived questions Jev reads 9 of the 10 traps that fool the rules, where the rules read none, so Jev's value on real questions still waits on the blind set in #1.

A fresh session works the open issues in the order in Section 08, starting with #12, gated by the suites, and #1 waits on about 50 blind questions from you.

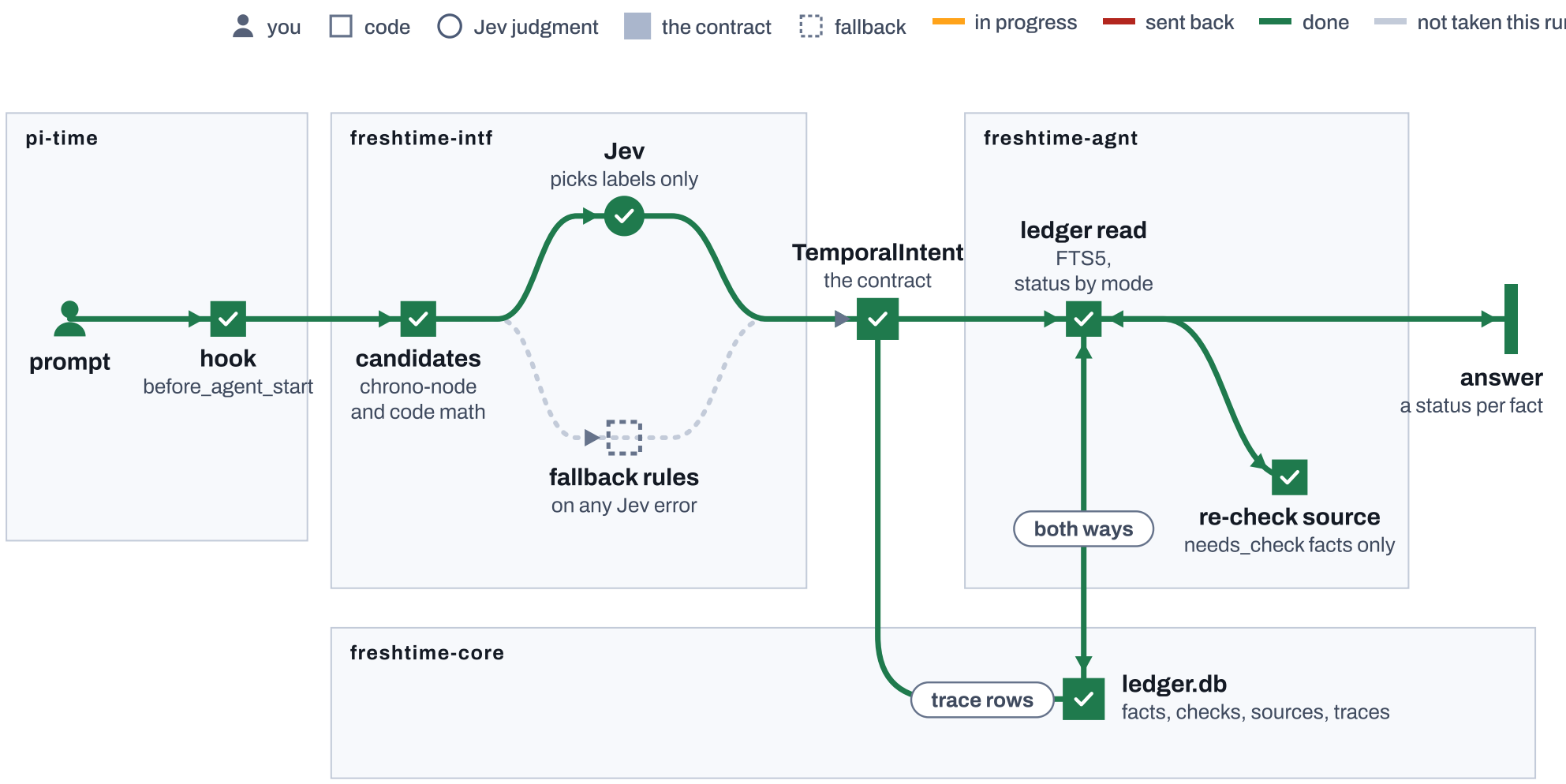
01 Architecture

A question passes through the four parts in one direction. freshtime-core holds the contract and the plumbing both tools share: the types, the SQLite schema, the trace writer, the clock and the Jev client. freshtime-intf turns a prompt plus conversation state into a `TemporalIntent`. freshtime-agnt reads the ledger at the time that intent names and re-checks stale facts against their source files. pi-time wires both into pi through one extension: a hook reads every prompt, and four tools expose the pair to the model.

Jev sits inside freshtime-intf as an optional judge, and a rule-based fallback answers whenever Jev is off or fails. Every parse, ingest, read and maintenance pass writes a row to the traces table inside the same ledger file.

FIG 1

One question through the four parts



T+0.0s	**you**	"What is the next step for alpha now?"	`[prompt]`
T+1.0s	**pi-time**	before_agent_start asks for a reading	`[hook]`
T+1.9s	**intf**	chrono-node finds no date phrase	`0 candidates [ok]`
T+2.7s	**Jev**	one batched call: which mode, and is this about right now	`current, now yes [jev ok]`
T+3.8s	**intf**	code applies the guards and builds the reading	`current, default basis [reading]`
T+4.8s	**pi-time**	reading goes into the system prompt and the status line, the model calls freshtime_ask	`[tool call]`
T+5.9s	**agnt**	text match over the facts, 4 are past their shelf life	`4 [needs_check]`
T+6.9s	**agnt**	re-reads their source files and re-ingests the changed ones	`2 of 4 [changed]`
T+8.1s	**agnt**	retrieves again after the re-ingest ran the replacement rules	`[current, replaced]`
T+9.2s	**core**	intf.parse and agnt.read write their trace rows	`[traced]`
T+10.1s	**answer**	alpha launched is current, the older Next: lines are replaced	`6 current, 2 replaced [statuses]`

An example run of the first question in the synthetic script, "What is the next step for alpha now?". The Jev step and the statuses follow the end-to-end run, where Jev read the question as current at confidence 0.98 and the answer held 6 current and 2 replaced facts. The re-read counts follow the ablation with the ledger carried, where the same question re-read 4 facts and caught 2 changes. The fallback path is drawn dotted because this run used Jev. The log times are the replay's clock, not measured latency.

02 freshtime-core: the contract

The contract is two records. intf produces a `TemporalIntent`, agnt consumes it and returns LedgerFacts, and dates are ISO calendar dates with no time zone.

TemporalIntent

The intent carries seven fields.

FIELD	VALUES	WHAT IT CARRIES
`asof_mode`	current, past, range, future, mixed, none	Which time the answer is about.
`anchor`	start, end and the words it came from, or null	The date or span. A point has start equal to end.
`basis`	stated, inherited, default	Said in this question, carried from earlier, or now.
`kinds`	a subset of the twelve kinds of time	Which kinds the question needs.
`confidence`	0 to 1	From Jev, or 0.6 on the rules.
`path`	jev, fallback, manual	Who made the reading, so a silent Jev failure shows.
`query`	the question with its time phrases removed	The text used for retrieval.

LedgerFact

A fact carries four clocks and three freshness fields.

GROUP	FIELDS	MEANING
Identity	`id`, `project`, `section`, `text`, `source_path`	One bullet or entry, where it lives.
Class	`kind`	Volatility class: identity, preference, decision or status.
Clocks	`valid_at`, `invalid_at`, `recorded_at`, `source_at`	True from, true until, learned on, file date.
Freshness	`last_verified_at`, `verified_by`, `last_accessed_at`	Last confirmed, by whom, last read.
Read view	`status`	current, replaced, `needs_check` or `current_then`, set at read time.

Statuses and shelf lives

STATUS	MEANING
`current`	Still true as far as the source shows.
`replaced`	A newer line or a re-read closed it. Never shown as current.
`needs_check`	Past its shelf life and not confirmed by a re-read.
`current_then`	True at the past date or span the question asks about.

Shelf lives are hand-set per class: status 7 days, decision 90, preference 365, identity never. A ponytail comment in the contract marks them for fitting from verification outcomes.

SQLite schema

The SQLite schema has five tables in one file, opened in WAL mode at `~/.freshtime/ledger.db` unless `FRESHTIME_DB` names another.

The schema is five tables.

TABLE	HOLDS
`facts`	One row per fact, unique on source path, section and text.
`facts_fts`	FTS5 index over project, section and text, filled by an insert trigger.
`checks`	One row per re-read: the fact, its age in days, whether it changed, and why.
`sources`	One row per source file: its hash and when it was last ingested.
`traces`	One row per tool call: tool, input, output, path, milliseconds and any error.

Traces, clock and the Jev client

Traces clip each input and output to 2,000 characters, and the newest rows feed `/freshtime` and the trace command line. `FRESHTIME_NOW` pins the clock, which is how tests, the replay and the end-to-end suite set dates.

The Jev client is one fetch to the TypeSafe System One endpoint, with the model pinned to `jev-1.13.0` unless `FRESHTIME_JEV_MODEL` says otherwise. It is on only when `TYPESAFE_API_KEY` is set and `FRESHTIME_JEV` is not off, and it throws on any failure instead of guessing, because every caller has a fallback. Jev answers two question types: a choice among labels with a confidence, and a yes-or-no probability.

03 freshtime-intf: temporal intent

Candidates in code, judgments by Jev

Code finds the candidates and does every date sum, and Jev only picks labels. `chrono-node` finds date phrases against the session clock. Code then turns "last week" or "next month" into Monday-to-Sunday weeks or whole months, widens a bare month to its full span, and merges "between X and Y" into one range.

The rules pick the mode from the candidate and a few word lists. A stated date in the future, or a future word, reads as future. "Since" or "what changed" reads as mixed. A span reads as range, and a single date as past. With no date, a now word reads as current, then a held as-of is inherited, then a future word reads as future. Anything else is none.

When Jev is on, one batched call asks up to four questions: which mode, whether the question is about now, which candidate phrase sets the time, and, when an as-of is held, whether this question changes the period. A choice counts only at confidence 0.5 or more, and a yes only at probability 0.5 or more. Jev never writes a date. It picks one of the candidates code found, or none. On any Jev error the rule reading stands and the error is stored on the intent and in the trace row.

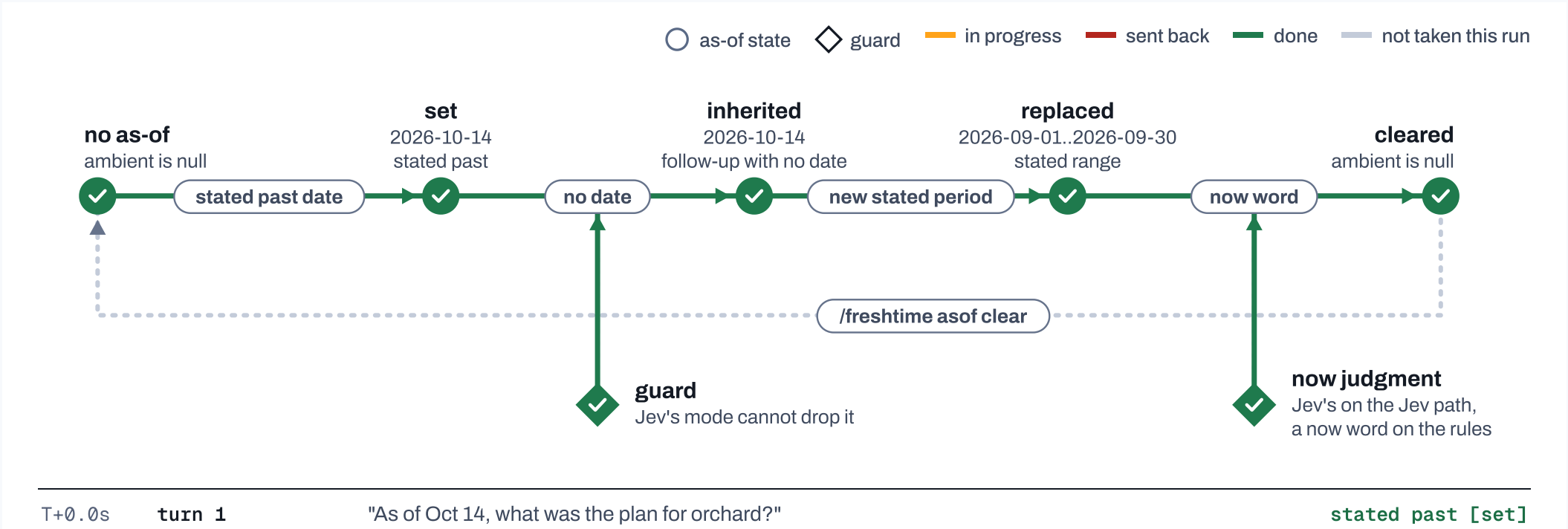
Ambient as-of

The ambient as-of is the period a conversation is in. A stated past date, span or since-question sets it. A follow-up with no date inherits it. A new stated period replaces it. A current reading clears it: on the rules that takes "now", "currently" or "today", and on the Jev path Jev's own now judgment. A Jev yes to "does this change the period" also clears it, and `/freshtime asof` sets or clears it by hand.

Guards and Jev's now judgment

Live Jev failures shaped how Jev and the rules combine. An inherited reading is kept: Jev's mode choice alone cannot drop it, and only the "does this change the period" judgment can. Live Jev had read the follow-up "And the next step for alpha?" as future, because of the word "next". A second guard once let an explicit now word override Jev's mode, after live Jev read "What is the next step for alpha now?" as future. The probe then showed that guard threw away three of Jev's right answers, such as "Was that still true at the time?", which has "still" and is about the past. Since commit `e9ef5f7`, on the Jev path "is this about right now?" is Jev's own judgment, and the keywords decide only on the fallback path. Jev still reads "next step" as future at 0.9 confidence while its own now judgment says now at 0.8, and the combination follows the now judgment.

FIG 2 **Ambient as-of across a four-turn conversation**



T+0.0s	**turn 1**	"As of Oct 14, what was the plan for orchard?"	stated past [set]
T+1.2s	**turn 2**	"and the next step for orchard?"	no date, inherited [kept]
T+2.4s	**guard**	Jev's mode choice alone cannot drop an inherited reading	[kept]
T+3.4s	**turn 3**	"and in September, what was the plan for orchard?"	stated range [replaced]
T+4.6s	**turn 4**	"what is the orchard plan now?"	now word, current [cleared]
T+5.8s	**now**	on the Jev path Jev's own now judgment decides, on the rules a now word does	[current]

The asof kinds test, replayed. Turn 1 states a date and sets the as-of, turn 2 inherits it, turn 3 states a new period and replaces it, and turn 4 asks about now and clears it. The diamonds are the inherit guard and the now judgment, and the dotted line back to the start is the hand reset with `/freshtime asof clear`.

04 freshtime-agnt: the freshness ledger

Sources, extraction and dating

Sources are `whats_next.md`, `HANDOFF.md`, `docs/handoffs/*.md` and `learning_guide*.html` under `~/create` and `~/repos`, at most four folders deep, skipping dot folders and `node_modules`. `FRESHTIME_ROOTS` replaces the two roots.

Markdown extraction takes each bullet or checkbox line of 12 characters or more outside code fences, under its nearest heading. Learning guides give one fact per definition term and per first table cell, under their section heading. A fact's project is the folder that holds its file, or the repo root for `docs/handoffs`.

Dating follows the nearest date the author wrote. In markdown that is the nearest dated heading, then a date prefix on the filename, then the file's modified date. In a learning guide it is the section heading's date, then the last date in the title, then the modified date. At ingest, `last_verified_at` is the author's date and `verified_by` is author, because reading a line is not checking it. Jev classifies each new line into a volatility class in batches of 40, and a keyword rule takes over on error.

Replacement rules

Four replacement rules close a fact at ingest.

The rules carry these letters in the code.

RULE	CLOSES	WHEN	CLOSED AT
(a)	An open checkbox	The same text appears checked in the same file.	The checked line's date
(b)	A `Next:` line	Its successor in the project: the next later-dated one, or the next one lower in the same file on the same day.	The successor's date
(c)	Any line	It is gone from its file on re-read.	The day it was found gone
(d)	A state-like section of a handoff	A later-dated handoff of the same project has the same section (State, Next step, Recent artifacts and similar).	The later handoff's date

A removed line that comes back is reopened, and the gap between removal and return is not kept.

Verification, reads and maintain

Verification is a re-read of the source file. If the file's hash changed since the last ingest, the file is re-ingested first so the replacement rules run. The fact is verified when its line is still there and no rule closed it: `last_verified_at` moves to today and `verified_by` to source. Otherwise it is closed today. Every re-read writes a checks row with the fact's age and whether it changed, which is the data shelf-life fitting needs.

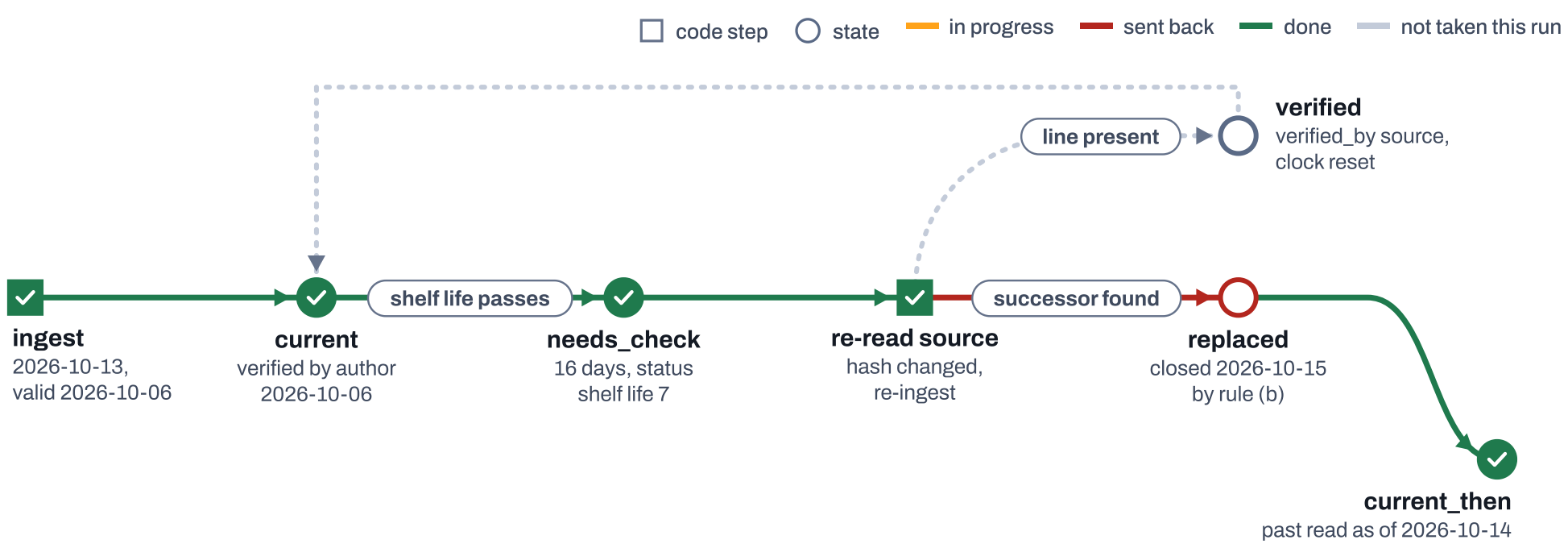
Each read takes a text-match pool five times the limit, then prefers live facts and newer ones. The status depends on the mode.

Three read shapes cover the six modes.

MODE	FACTS RETURNED	STATUS	RE-READS
current, none, future	All matches	`replaced if closed, needs_check if past shelf life, else current`	`needs_check` facts
past, range	Valid on or before the anchor's end and not closed before its start	`current_then`	None
mixed	The past set at the anchor, plus live facts dated after it	`current_then for the past set, current rules for the rest`	The newer facts only

When a re-read catches a change, the read runs once more without re-reading, so facts the re-ingest added can answer. Maintain re-checks stale live facts up to a budget, most recently read first, then the longest unconfirmed.

FIG 3 **One fact from ingest to replacement**



T+0.0s	**ingest**	alpha/whats_next.md read on 2026-10-13, author date 2026-10-06	**[current]**
T+1.1s	**clock**	2026-10-22, 16 days since the author wrote it	**[needs_check]**
T+2.2s	**agnt**	a now question re-reads the source, its hash changed	**[re-ingest]**
T+3.3s	**rule (b)**	the 2026-10-15 Next: is its successor	**closed 2026-10-15 [replaced]**
T+4.5s	**read**	as of 2026-10-14 it was valid and not yet closed	**[current_then]**

The fixture fact "Next: alpha needs a load test before launch", written in the 2026-10-06 session and ingested on 2026-10-13. By 2026-10-22 it is past the 7-day status shelf life, the re-read finds a 2026-10-15 session appended, rule (b) closes it at its successor's date, and a past read as of 2026-10-14 still returns it as `current_then`. The verified branch is dotted because this fact changed.

05 pi-time

Launcher and agent directory

The launcher sets `PI_CODING_AGENT_DIR` to `pi-time/agent`, so pi-time keeps its own settings, sessions and extension apart from the user's pi. It links `auth.json` and fd from `~/.pi/agent` instead of copying them, installs `chrono-node` if it is missing, and starts pi with `--no-skills` so shared skills stay out. The agent settings pin provider zai, model `glm-5.3` and project trust never.

Tools, per-turn reading and status line

Four tools reach the model.

TOOL	DOES
`freshtime_ask`	Runs the pair: the turn's reading, the ledger read and the re-checks, a status per fact.
`temporal_intent`	Reads the time reference only, and updates the held as-of like a turn.
`ledger_query`	Reads the ledger with an explicit mode and anchor, to override the reading.
`ledger_verify`	Re-reads one fact's source and returns its status before and after.

Before each turn a hook parses the prompt, adds the reading and three rules to the system prompt, and updates the status line. The rules are to call `freshtime_ask` for project questions, say the reading back in one line, and never present a replaced or `needs_check` fact as current. The status line reads "ft <facts> facts | <stale> stale | reading: as of 2026-10-14 (inherited from earlier) | jev ok", and the last field turns to "jev FALLBACK n" after any Jev error in the session. `/freshtime` shows counts by status and class, the held as-of, the top projects, the last 10 traces and the remaining work. Its subcommands are ingest, maintain, asof and trace.

Startup and remaining work

On startup pi-time builds the ledger if it is empty, or re-ingests only the files whose hash changed, then lists remaining work. The list is the open GitHub issues, needs-input first, then `agent:todo`, then a count of ideas, with `REMAINING.md` as the offline fallback.

Observability and privacy

Every Jev failure is loud. The trace row keeps the error, the status line counts fallbacks, `/freshtime trace` marks the row ERROR, and the answer still arrives on the rules. A test hook, `FRESHTIME_TRACE_FILE`, writes each turn's reading and each `freshtime_ask` result as one JSON line. Without a terminal UI the extension also writes to stderr, and inside the UI it never does, because stderr lines tear the pi screen.

The repo is public. The ledger lives outside it, and sessions, the auth link, the fd link, model store and trust file are gitignored. Fixtures are synthetic, replay reports hold counts and rates only, and issue bodies name engineering work, not corpus content.

On the real corpus

On today's corpus a build with keyword classes finds 70 source files, 13 of them learning guides, and 1,914 facts, of which 49 are replaced and 278 are past their shelf life. Keyword classes label most lines decision, so a build with Jev classes, as pi-time runs it, finds more status facts and more stale ones.

06 Tests

Six layers each answer a different question.

Latest results by layer, with the run each came from below.

LAYER	PROVES	CANNOT PROVE	LATEST RESULT
Unit `npm test`	Extraction, dating, the four rules, statuses, verify, maintain, the intent rules and fallback.	That the rules match how people write or ask.	22 pass (intf 9, agnt 13)
Ablation `node test/ablation.ts`	The mechanism: what each tool adds, and that the pair removes all harm on the script.	A rate on real data. Seven questions, three synthetic projects.	both 100% right status, 0 harm, 2 assertions pass
Twelve kinds `npm run kinds`	Which kinds of time are built, by a method per kind over generated git histories.	That a passing kind works on real docs.	25 pass, 11 todo, 0 fail, plus 2 generator tests
Replay from git `npm run replay -- --jev`	Dating and past reads against git truth on real history, and how well shelf lives predict change.	Over-reach by rule (d), and anything about questions.	recall 0.998, precision 1.0 without backdated lines
End to end `npm run e2e -- --runs 2`	Real pi and `glm-5.3` call the tools, read each turn right, and answer without harm.	Behavior beyond seven scripted turns, or a stable pass rate from 2 runs.	14 of 14 turns over 2 runs, fallback 7 of 7 answered
Jev probe `npm run probe`	Jev is not echoing the rules, because Jev never sees the rules' answer, and where the combination loses what Jev had right.	Accuracy. The 21 questions are contrived against the rules by design.	traps: rules 0 of 10, Jev 9, combined 9

Where each number comes from

- `npm test` runs the unit, ablation and twelve-kinds tests offline: 62 tests, 51 pass, 0 fail and 11 todo, with Jev forced off in every test file. `whats_next.md` records 50 pass and 12 todo. The count moved because commit `5c375b9` dropped a todo marker on a test that had passed since the successor fix.
- Unit, ablation, kinds, score and probe numbers were re-run on 2026-10-22 at 01:23 at commit `e9ef5f7`.
- The end-to-end numbers come from `$TMPDIR/freshtime-e2e/2026-10-22T05-19-13-262Z`, whose results were written at 01:23, and the fallback numbers from the 04-55-19-266Z-fallback run, written at 00:56. The commit note for `e9ef5f7` records the end-to-end result as unchanged after that commit's change to how Jev and the rules combine.
- The fallback run and the replay came before `e9ef5f7`. The replay exercises only the ledger, which that commit did not touch. The replay numbers come from

`~/.freshtime/replay/2026-10-22/report.md`, a 133-second run with Jev classes.

Coverage matrix

Each kind's claim, from the test plan, with today's counts.

KIND	CLAIM	PASS	TODO	WHAT THE TODO WAITS ON
`event`	A period question returns facts dated in it	1	1	In-text event dates
`valid`	A replaced fact answers the past, not now	4	1	#7
`record`	The ledger knows when it learned a fact	2	1	Record-time as-of
`source`	An old doc imported today does not look new	1	1	Git dates
`reference`	Relative phrases resolve to the right date	5	1	Relative phrases inside docs
`sequence`	A later same-day write wins	1	1	Same-day handoffs never replace (#12)
`verification`	Only the author or a re-read confirms	3	1	#3
`access`	Recently used stale facts are checked first	1	0	-
`volatility`	Shelf life follows the class	3	1	#4
`intended`	A future commitment is captured	1	1	#8
`business`	Fiscal periods map to dates	2	2	#9
`asof`	A stated date carries until changed	1	0	-

A todo is a real assertion that fails today and flips when the feature lands.

Ablation

The ablation runs with the intent rules and the ledger carried through the script, and again cold with a fresh ledger per question.

Ablation totals over the seven questions. Found is the share of gold facts returned, and harm counts harmful facts shown as true.

SETUP	WHAT RUNS	FOUND	RIGHT STATUS	HARM	RE-READS CARRIED	RE-READS COLD	CAUGHT COLD
none	Text search, read as now, no checks	79%	36%	4	0	0	0
intf	Intent reading, plain store, no freshness	79%	79%	2	0	0	0
agnt	Full ledger, every question read as now	86%	57%	4	4	24	11
both	Intent reading and the full ledger	100%	100%	0	4	9	4
interaction	-	+0.14	0.00	-2	-	-	-

Harm is the count of harmful facts shown as current or `current_then`, so the interaction 0 - 2 - 4 + 4 = -2 means the pair removes 2 more than the two tools remove separately. The ledger alone removes no harm because it re-checks, finds the newest fact, and then shows it as true for questions about the past. Found facts show a positive interaction too, +0.14: only a re-read finds the two facts appended after ingest (q1 and q5), and only an as-of read returns the fact that was current on Oct 1 (q2). On the carried script one early now question pays for the re-reads later questions use, so both setups spend 4. The cold run is the fair comparison.

Intent score, replay and end to end

The intent score is a regression check: the rules get 20 of 20 and Jev 19 of 20 on `asof_mode`, both 20 of 20 on anchor and basis, at 2 ms and 112 ms per question. The labels were written next to the rules, so it does not measure accuracy.

The replay on real history scores the ledger against git, not against its own rules. Over 50 sources and 143 sample dates, recall against git truth is 0.998 and precision is 1.0 once 345 backdated lines are set aside. 22 lines were misdated. The freshness check over 479 version pairs shows the 7-day status shelf life flags 4,672 facts of which 14% changed, about 7 flagged for each real change, and it catches 61% of status changes. The share of status facts changed rises with age: 7% at 0 to 7 days, 12% at 8 to 30, and 35% at 31 to 90. No fact in the sample is older than 90 days, so the decision and preference shelf lives never fire.

The end-to-end suite drives real pi with `glm-5.3` through the seven questions as one conversation and checks the tool call, the reading, the facts and three Jev-judged checks per turn. The latest run passed 14 of 14 turns over 2 runs at about $0.06 per run as pi reports cost. With a bad Jev key all 7 turns still answered, and 8 of 8 trace rows recorded the error.

The Jev probe scores the rules alone, Jev's raw mode choice alone, and the combination on 21 contrived questions. On the 10 traps worded to fool the keyword rules, the rules get 0, Jev alone 9 and the combination 9. All three get the 6 controls right. On the 3 questions with no time in them the rules get 3 and Jev 2, because Jev read "What color is the alpha logo?" as current. On the 2 questions about a past with no date, such as "What did we believe before the launch?", Jev gets both and the combination neither, because the contract has no way to say "past" without an anchor date. That gap is filed as #13.

07 Progress

Built on this branch and checked by the suites in Section 06: the contract, both tools, the pi-time launcher and extension, the ablation, and three test suites, with remaining work filed as issues. Each defect below was found by running the system.

Defects found and fixed, in the order they were found.

#	DEFECT	FOUND BY	FIX
1	Every fact read as verified today, so the status line showed 0 stale	First interactive run, in a herdr pane	`last_verified_at` at ingest is the author's date
2	No real fact was ever replaced, because real handoffs use section headings	First build on the real corpus	Rule (d) and same-day `Next:` lines
3	Text ranking answered a "current state" question from an older handoff	A question on the real corpus	A 5x text-match pool, then live and newer facts first
4	Shared skills loaded with parse warnings, and pi downloaded fd into the repo	First launch	`--no-skills`, and links to the user's `auth.json` and fd
5	Stderr lines tore the pi screen	Herdr side pane run	Stderr only when there is no UI
6	Jev dropped an inherited as-of on a follow-up	Jev ablation	Guard: only the change-of-period judgment drops it
7	One ingest of three `Next:` lines closed all of them at the newest date	Kinds suite, valid	Each closes at its successor's date
8	A removed line that came back stayed closed	Replay suite	Reopen it
9	Jev read "next step ... now" as future	End-to-end suite	Guard: an explicit now word wins, replaced by row 13
10	The judge failed right historical answers that mentioned the present	End-to-end suite	Ask "true for the asked time", not "currently true"
11	The first replay counted rule replacements of present lines as misses	Reading the first replay report	A separate bucket for rule claims
12	A todo marker stayed on a test that passes	Writing this page	Marker dropped in commit `5c375b9`
13	The now-word guard threw away three of Jev's right answers	Jev probe	On the Jev path, Jev's own now judgment decides

Three counts behind these rows were recorded in the learning guide and not re-run here: 0 of 809 facts stale before the verification fix and 342 after, 0 of 809 replaced before rule (d) and 48 after, and 1,998 replay misses from the closed re-added lines. One more issue was handled without a code defect: a ledger with the real corpus appeared in the home folder during the worker runs, its cause untraced, and it was deleted so the first real run stays a first run.

08 What's next

Seven issues are open for agents or for you, and five more are parked ideas. The order below puts first what changes another issue's measurement.

The open issues in the order that unblocks most.

ORDER	ISSUE	WHAT IT DOES	WHY IT GOES HERE	GATE
You, now	#1	About 50 intent questions written blind by you	The only issue that needs you. It decides whether Jev stays on the intent path, and nothing else waits on it.	`node freshtime-intf/score.ts`
1	#12	Scope rule (d) to handoffs of the same unit, and break same-day ties	Rule (d) supersedes 6,416 line-dates in the replay, more than any other rule, so it moves every replacement count downstream.	`npm run replay -- --jev`, kinds sequence todo flips
2	#7	Replace other status bullets beyond `Next:` lines and sections	Same replacement code as #12, so the rules change once and are measured once.	Kinds valid todo flips, replay
3	#6	Match reworded facts	A reworded bullet reads as replaced plus new, which loses its verification history and counts as a change in the freshness data.	`npm test`, replay
4	#5	Classify re-ingested facts with Jev	Facts added during a re-read get keyword classes, and shelf lives are fitted per class.	`npm test`, ledger counts by class path
5	#4	Fit shelf lives from the checks table	Needs the rules, fact identity and classes above to be stable first. The data is already on the issue.	Kinds volatility todo flips, replay
6	#3	Record verifications and replacements stated in chat	Independent. It adds `verified_by` user as a new kind of check.	Kinds verification todo flips, `npm run e2e`

The recommendation is a fresh session that takes this queue one issue at a time, with `npm test` at 0 fail before each commit, the replay re-run after #12, #7, #6 and #4, and `npm run e2e -- --runs` 2 once at the end. A todo that flips becomes a plain assertion in the same commit, so the suites tighten as the work lands.

09 What's after

Beyond the open queue, each item is a decision with a cost on each side.

Decisions after the open queue.

DECISION	CHOICE	FOR	AGAINST	DECIDE WHEN
Store adapters (#10)	Put Graphiti or Mem0 behind the contract	Entity resolution and scale without changing intf	Another service, and each engine has its own idea of current that must map to four statuses	The ledger needs sharing across agents or outgrows SQLite
qmd retrieval (#11)	Let qmd supply the candidates	Could find reworded matches that token-OR text match misses	A second index to keep in step, and a ranking blend to tune	Replay or e2e show retrieval misses, not dating misses
Shelf-life curves	Fit change by age, not one cut-off	The replay shows change rising with age, which a single cut-off ignores	The data is handoff files only and correlated within a source, so a curve may fit habits, not truth	#4 has run and the checks table has months of rows

DECISION	CHOICE	FOR	AGAINST	DECIDE WHEN
Intended time (#8)	Schedule what the reading parses	Closes the one kind that is parsed and then dropped	Needs a scheduler and a way to fire inside pi. The research rates schedulers as available and the link to memory as the gap	A real reminder use case appears
Fiscal calendars (#9)	Map FY phrases to dates	"FY26" and "Q3 FY26" get anchors instead of only a kind tag	A per-organization setting to keep correct	Questions about fiscal periods show up
Past with no date (#13)	Resolve "before the launch" to the date of a ledger fact, or ask for a date	Jev already reads these as past, so the reading would follow	A new anchor kind in the contract, and an event match that can pick the wrong fact	Blind questions in #1 show how often people ask this way
Jev on intent	Keep Jev on the intent path	On the probe's 10 traps Jev reads 9 and the rules none	No gain on the labeled sets, one guard still needed, a known "next step" weakness, and 112 ms against 2 ms per question	After the blind set in #1

Jev's value on intent is the open question with the most riding on it. On the labeled sets its reading adds nothing over the rules, and on the contrived traps it reads 9 of 10 that the rules miss. Neither set was written blind, so #1 decides it. Classification is a different use: Jev labels each line's volatility, and keyword classes put almost everything into decision.

10 Risks and limits

- The ablation and end-to-end suites run on synthetic fixtures: seven questions over three projects, two edits behind the ledger. They show the mechanism, not a rate.
- The 20 intent labels were written next to the rules, and the ablation gold was written with its fixtures for this test, so passing either is a regression check.
- The replay samples are thin: 50 sources, 143 sample dates and 479 version pairs, correlated within each source, with no confidence intervals and no fact older than 90 days.
- The replay's rule (d) over-reach check is inconclusive: it flags all 1,842 cases, because the filename test passes whenever two handoffs have different names and cannot tell a new unit of work from the same unit renamed.
- Two end-to-end runs do not give a stable pass rate, and the suite's own notes name three sources of judge and model noise.
- The fallback end-to-end run predates commit `e9ef5f7`, which also reordered the rules so that a held as-of now wins over a future word.
- The 7-day status shelf life is costly on real history: it flags 4,672 facts of which 14% changed, about 7 re-reads for each change it finds.
- Project text leaves the machine: ingest sends each new line to Jev, TypeSafe's hosted API, in batches of 40 for classification, and pi-time sends questions and ledger facts to the zai model. `FRESHTIME_JEV=off` keeps classification and intent reading on the local rules, and the model calls remain.

- The probe is contrived against the rules by design, so its scores show that Jev is not echoing the rules and say nothing about accuracy.

Built 2026-10-22 from the repo at commit `e9ef5f7`. Numbers carry their run in Section 06, and the command outputs are in `docs/content/freshtime-design/runs-2026-10-22.txt`.